UNISENAI

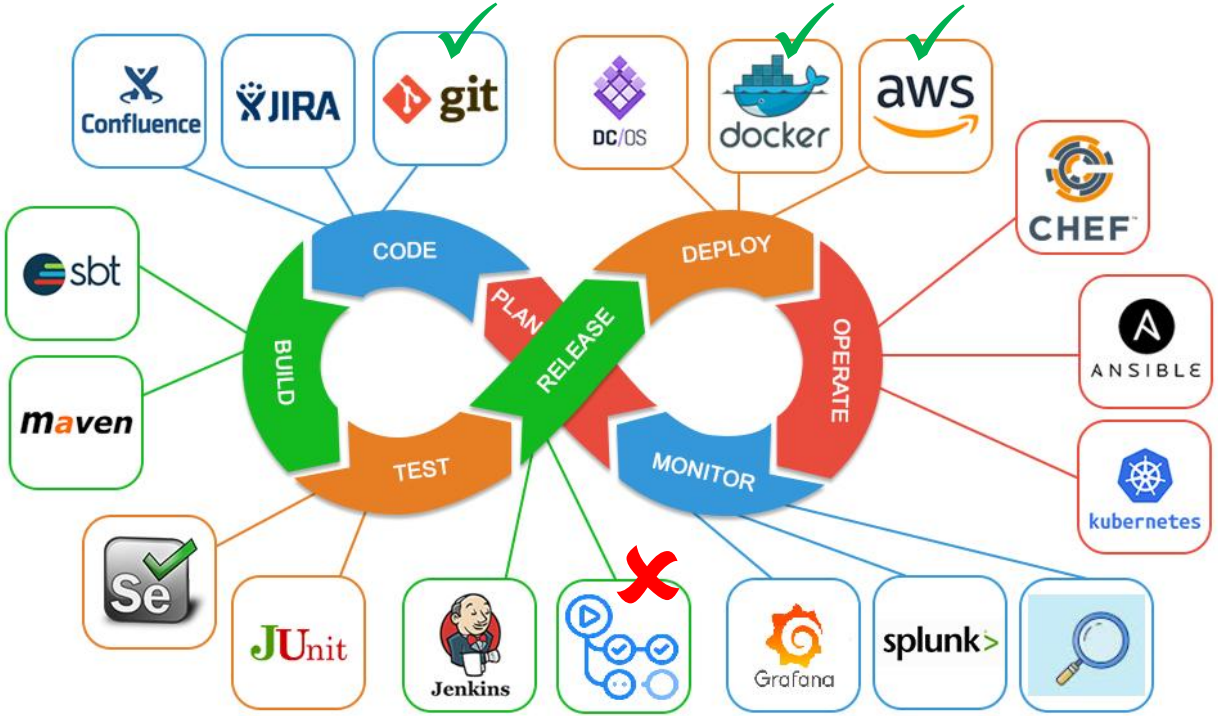
CI/CD

Do Código à Produção com Confiança

Integração Contínua • Entrega Contínua • Deploy Contínuo

`< Automation is the key />`

ONDE ESTAMOS EM NOSSA JORNADA...



Você já travou uma sexta às 17h tentando fazer deploy?

"Funcionava na minha máquina!"

"O build quebrou, quem fez o commit?"

"Rollback! Rollback! O site caiu!"

Quem já usa CI/CD?

Empresas que transformaram entregas com automação

Amazon

136k

deploys por dia

Um deploy a cada 11,6 segundos. Equipes autônomas, pipelines independentes e rollback automático.

Netflix

100+

deploys diários

Chaos Engineering + CI/CD. Cada equipe mantém seu próprio pipeline com testes automatizados.

Spotify

< 5min

do commit ao deploy

Modelo Squad com ownership total do pipeline. Feature flags permitem deploys sem downtime.

CI Integração Contínua

Prática de mesclar código frequentemente ao repositório principal, com verificação automática a cada commit.

01 Desenvolvedor faz commit

02 Trigger automático dispara

03 Build do código

04 Testes automatizados rodam

05 Feedback imediato ao dev

Princípios Fundamentais

Commits frequentes
Pelo menos uma vez por dia ao branch principal

Build automatizado
Ninguém faz build manual — a pipeline faz tudo

Testes obrigatórios
Unit, integration e e2e rodam em cada push

Feedback rápido
Resultado em minutos, não horas ou dias

Build quebrado = prioridade
Ninguém continua enquanto o build estiver vermelho

CD — Duas Definições Importantes

Continuous Delivery

Entrega Contínua

O código está SEMPRE em estado deployável. O deploy em produção exige aprovação humana.

✅ *Indicado para sistemas críticos onde deploy requer validação extra*

Continuous Deployment

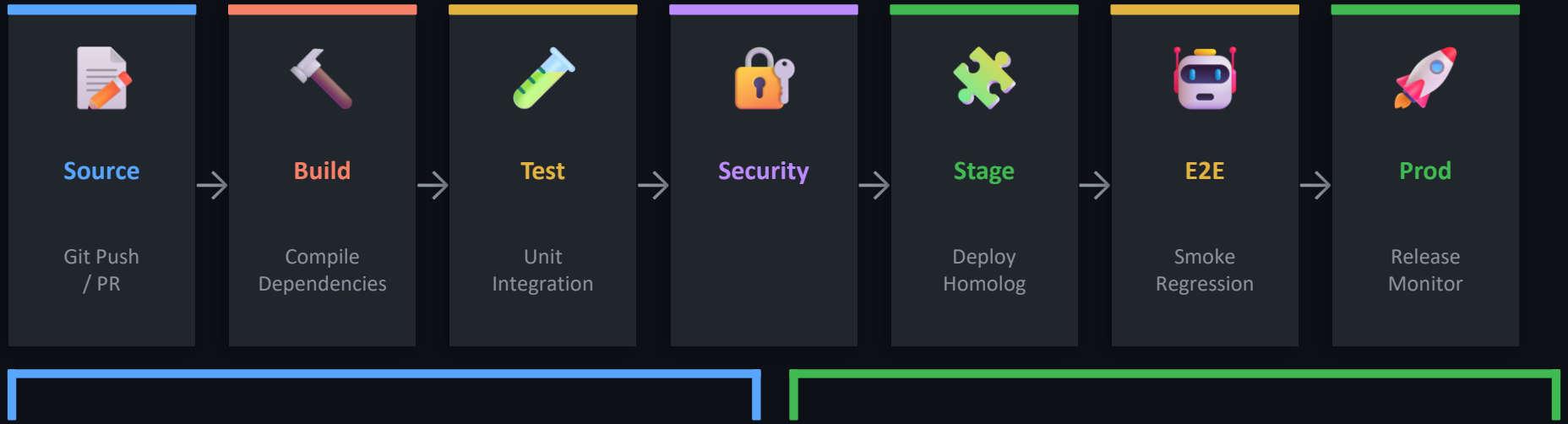
Deploy Contínuo

Cada commit que passa em todos os testes vai AUTOMATICAMENTE para produção. Zero intervenção.

⚡ *Indicado para equipes maduras com alta cobertura de testes*

O Pipeline Completo

Do commit ao usuário final



Pipeline CI/CD Completo

Por que adotar CI/CD?

10x

Velocidade

entregas mais rápidas

Ciclos de dias/semanas passam a horas. Time-to-market drasticamente menor.

~80%

Qualidade

menos bugs em prod

Problemas detectados antes de chegar ao usuário. Custo de correção muito menor.

24/7

Confiança

deploys sem medo

Deploy deixa de ser um evento estressante e passa a ser rotina automática.

100%

Visibilidade

rastreabilidade

Cada mudança é auditável. Saber o que, quando e quem fez o quê.

< 5min

Rollback

para reverter

Incidentes resolvidos em minutos. Versão anterior sempre disponível.

↓ 40%

Custo

custo operacional

Menos trabalho manual, menos erros, menos horas de incidente.

Ferramentas do Ecossistema CI/CD

Orquestração de Pipeline

GitHub Actions
Nativo ao GitHub, YAML-based, marketplace rico

GitLab CI/CD
Integrado ao GitLab, runner self-hosted

Jenkins
Open-source, altamente customizável, Groovy DSL

CircleCI
Cloud-first, rápido, paralelismo nativo

Containerização & Deploy

Docker
Empacota apps em containers portáveis

Kubernetes
Orquestra containers em escala

Helm
Package manager para Kubernetes

ArgoCD
GitOps para Kubernetes declarativo

Qualidade & Segurança

SonarQube
Análise de qualidade e cobertura de código

Snyk / Trivy
Scan de vulnerabilidades em deps e imagens

Selenium/Playwright
Testes E2E automatizados de UI

Prometheus/Grafana
Monitoramento e alertas pós-deploy

Exemplo Prático: GitHub Actions

Anatomia de um workflow

Workflow (.yml)

Arquivo de configuração na pasta .github/workflows/

Trigger (on:)

Define quando o workflow roda (push, PR, schedule...)

Job

Grupo de steps que roda em uma máquina (runner)

Step

Comando individual ou action a ser executado

.github/workflows/ci.yml

```yaml
name:
  CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
      - run: npm ci
      - run: npm test
      - run: npm run build
```

Workflow Completo: Node.js → Docker → Deploy

Checkout → Test → Build Docker → Push Registry → Deploy Prod

.github/workflows/deploy.yml

```yaml
name: CI/CD Pipeline — Node.js

on:
  push:
    branches: [main]

env:
  IMAGE: ghcr.io/${{ github.repository }}

jobs:
  test:  # ── JOB 1: Testes ──
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test

  build-push:  # ── JOB 2: Docker ──
    needs: test
    steps:
      - uses: docker/build-push-action@v5
        with: push: true  tags: ${{ env.IMAGE }}

  deploy:  # ── JOB 3: Deploy ──────
    needs: build-push
    steps:
      - run: ssh user@server docker pull && restart
```

CI/CD não é sobre ferramentas.

É sobre cultura de qualidade e entrega contínua de valor.

1. Comece pequeno: um workflow simples de CI já muda tudo
2. Invista em testes: eles são a fundação de tudo
3. Automatize o que é repetível, monitore o que importa
4. Falhar rápido em dev é mil vezes melhor do que em prod

HANDS-ON

Hora do Código

```
user@cloud:~$mkdir meu-primeiro-servidor
```

UNISENAI

DEPARTAMENTO REGIONAL DE SÃO PAULO

www.sp.senai.br